

Interview Task 003: Developer Sandbox - Candidate Brief

Technical Assessment: AI Agent Sandbox

Role: DevOps Engineer

Company: Intrahealth (a subsidiary of HEALWELL AI)

Format: Take-home exercise + presentation

Time budget: ~2 hours (take-home) + 15–20 min presentation

The Challenge

You are joining a healthcare software company that builds an Electronic Health Record (EHR) platform. The engineering team uses AI coding agents extensively — agents that take tickets and produce tested pull requests. These agents need a sandboxed environment where they can clone repositories, build code, run tests, and validate output without affecting other work or the host machine.

Your task is to **build a Docker-based AI agent sandbox** that supports this workflow. The sandbox must be able to build and run two real-world open-source projects with very different technology stacks. You will need to make architectural decisions about how to handle those differences, and those decisions are a core part of what we're evaluating.

To be clear: we're not asking you to build an AI harness. We're asking you to build the **infrastructure that an AI harness runs inside**. The sandbox is the execution environment — the container (or containers) where generated code gets built, tested, and validated before it becomes a pull request.

What Is an AI Agent Sandbox?

An AI Agent sandbox is a reproducible, isolated environment where code can be built, run, and tested. In our context, these sandboxes are consumed primarily by AI coding agents, not by human developers sitting at a keyboard. That distinction matters — the sandbox needs to support non-interactive execution, deterministic builds, clean state between runs, and structured output capture.

Think of it as the difference between "I have Docker on my laptop and I can run stuff" and "I have a sandboxed environment that an automated pipeline can spin up, execute arbitrary

build/test cycles inside, capture structured results from, and tear down cleanly — hundreds of times a day."

The Two Projects

Your sandbox must successfully build and run both of the following:

Project 1: eShopOnWeb

A reference [ASP.NET](#) Core application demonstrating a monolithic web application with clean architecture.

- **Repository:** github.com/NimblePros/eShopOnWeb
- **Stack:** [ASP.NET](#) Core / C# / SQL Server / MVC + Blazor WebAssembly
- **What "works" means:** The application builds, database migrations run, seed data loads, the web application starts and responds to HTTP requests, and the existing test suite passes.

Project 2: Medplum

An open-source, FHIR-native healthcare platform.

- **Repository:** github.com/medplum/medplum
- **Documentation:** medplum.com/docs
- **Stack:** TypeScript / Node.js / PostgreSQL + Redis / React
- **What "works" means:** The server builds and starts, database migrations run, the API responds to health checks, and the test suite passes.

We chose these two projects intentionally. They have conflicting database requirements (SQL Server vs. PostgreSQL), different runtimes (.NET vs. Node.js), different build systems (dotnet CLI vs. npm/Turborepo), and different architectural patterns. How you reconcile these differences — one sandbox or two, shared services or isolated, layered images or separate compositions — is the interesting part.

What We Provide

The two repository links above. That's it.

Everything else — understanding the build requirements, figuring out database dependencies, reading existing Docker configurations (both projects have some), and deciding on your architecture — is part of the exercise. How you navigate unfamiliar projects and their infrastructure needs is something we want to observe.

Design Considerations

You don't need to address all of these, but they represent the kinds of decisions we want to see you reason about. Pick the ones that matter most and go deep.

1. **One sandbox or two?** — Do both projects share a single container/composition, or do they get isolated environments? What are the tradeoffs? What would an AI agent harness prefer?
 2. **Database strategy** — eShopOnWeb wants SQL Server. Medplum wants PostgreSQL (and Redis). Do you run both database engines? Consolidate on one? How do you handle schema isolation?
 3. **Clean state** — An agent runs a build, generates some code, runs tests. Then the next agent run needs a fresh environment. How does your sandbox support reset/rebuild without starting from scratch every time?
 4. **Non-interactive execution** — AI agents can't answer prompts, click through wizards, or manually accept licenses. Everything must run headlessly. Where do these projects have interactive assumptions and how do you handle them?
 5. **Build performance** — Layer caching, dependency pre-installation, multi-stage builds. A sandbox that takes 15 minutes to spin up is useless for an iterative agent workflow. What's your startup time strategy?
 6. **Output capture** — When an agent's code builds and tests run inside the sandbox, the results need to come back as structured data (not just a scrolling terminal log). How does your sandbox support this?
 7. **Resource management** — Memory limits, disk usage, CPU allocation. SQL Server alone wants 2GB+ of RAM. What are the resource implications of your design?
 8. **Security and isolation** — The sandbox will execute arbitrary code generated by AI agents. What isolation boundaries exist? What can code inside the sandbox access?
-

What You Deliver

This is intentionally open. You decide how to spend your 2 hours. We will evaluate the depth and quality of what you produce relative to the time invested.

Two valid approaches:

- **Implementation-heavy:** A working Docker setup (Dockerfiles, docker-compose, scripts) that builds and runs both projects. Accompanied by a README explaining your decisions and what you'd improve.
- **Design-heavy:** A detailed architecture document describing the full sandbox design — container topology, networking, storage, lifecycle management — with a partial prototype

demonstrating the core pieces. Justify your decisions thoroughly.

Your deliverables:

1. **The sandbox** — A link to a GitHub repo containing your Docker configuration, scripts, and documentation.
2. **Proof it works** — Evidence that both projects build and run inside your sandbox. Screenshots, test output, health check responses — whatever demonstrates it.
3. **A walkthrough (15–20 min)** — Present your work in the format that best showcases it. Screen-sharing a live demo of spinning up the sandbox and running both projects is ideal, but a recorded video or slide deck works too. The format is your call — we care about clarity of thinking and communication, not production quality.

A Note on AI Usage

We explicitly want to see how you use AI tools during this exercise — and we want you to tell us about it. Use Claude Code, Copilot, ChatGPT, Cursor, or whatever you normally work with.

If you use an AI agent to help build the sandbox, that's great — but notice what you're doing: you're using an agent to build the environment that agents will eventually run inside. What did the agent get right? What did it get wrong? Where did you have to intervene? That experience is directly relevant to the role.

What We're NOT Evaluating

- Expertise in either eShopOnWeb or Medplum (look everything up — that's expected)
- Whether every edge case is handled (2 hours isn't enough)
- Visual polish or fancy documentation
- FHIR knowledge or .NET mastery

Getting Started

Some practical guidance for your 2 hours:

- **Check what already exists.** Both projects have existing Docker configurations. Start there. Understand what they do, what they assume, and where the gaps are.
- **Get something running first.** A working sandbox that only handles one project is more valuable than an elaborate design for two that doesn't boot.

- **Document your decisions.** We care as much about *why* you made a choice as whether it works. A pragmatic tradeoff explained well beats an over-engineered solution without context.
- **Think about the consumer.** Your primary user is a human developer, but this sandbox will also be used in an automated pipeline. What does that change about your design?

Questions? If anything about the expectations is unclear, ask. We'd rather you spend time on the interesting problems than guessing what we want.

Good luck. We look forward to seeing your approach.